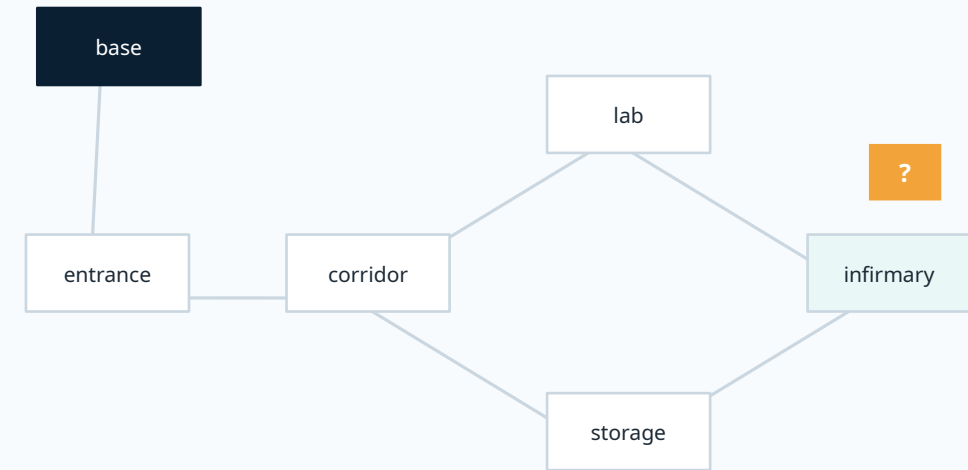


D4-V2 Search and Rescue

Unknown Victim Location

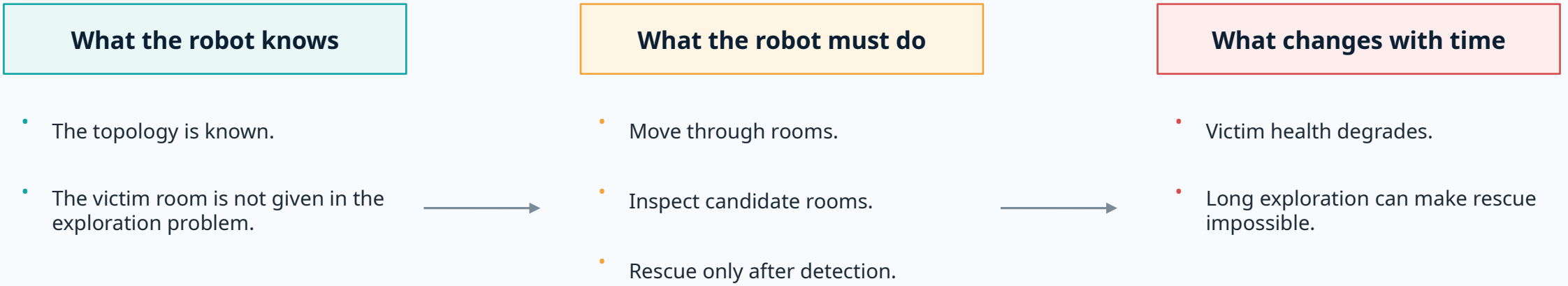
A rescue robot explores a known topology, inspects rooms, detects the victim, and completes rescue before time-dependent health degradation makes the mission fail.



Problem overview → classical abstraction → PDDL+ dynamics → model evolution → validation → limitations

Core modelling problem

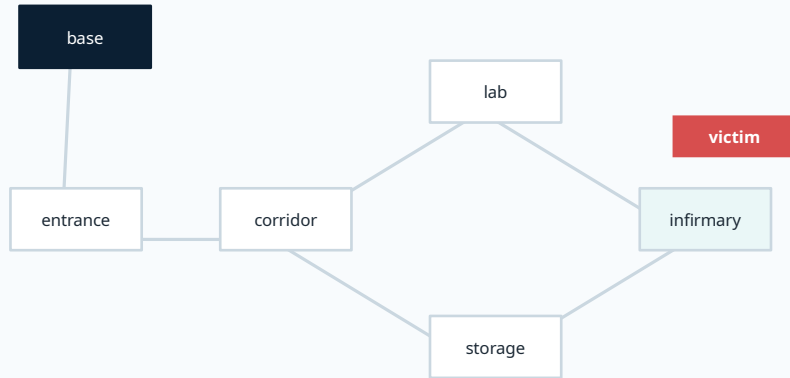
Unknown victim location forces the model to represent information acquisition explicitly.



Key modelling choice: treat sensing as explicit symbolic actions in Q1, then add time-dependent health dynamics in Q2.

Q1 — Classical PDDL approximation

Classical PDDL provides a valid abstraction, not real partial observability.



Symbolic information flow



rescue depends on victim-detected

direct rescue is blocked

- Rooms become inspectable symbolic states.
- The planner does not access hidden uncertainty directly.
- Detection changes the state, enabling the rescue action.

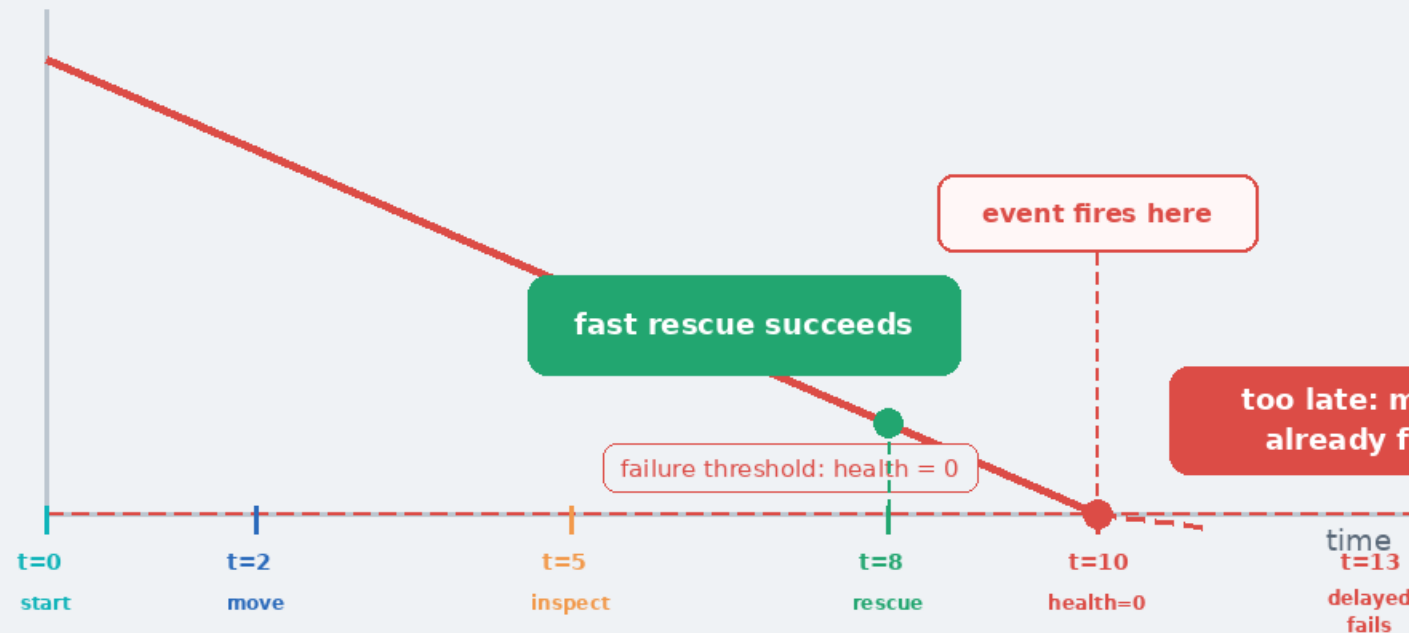
Presentation answer: Q1 approximates unknown location by making inspection and detection explicit in the domain model.

Q2 — PDDL+ adds time pressure

The world evolves between actions: victim health decreases while the mission is active.

Health over mission time

victim-health



**:process
health-degradation**

decreases victim-health
while mission-active
and victim-alive

**:event
victim-dies**

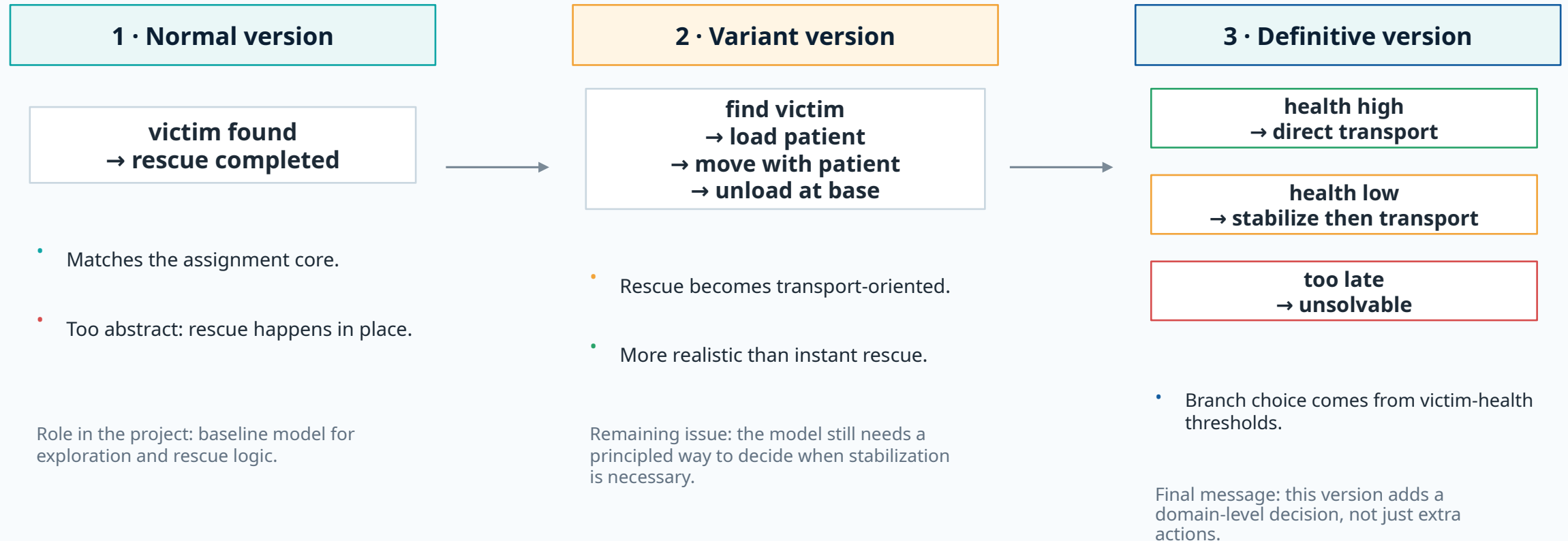
triggered automatically
when victim-health
reaches the threshold

**doing nothing is no longer neutral;
the planner must reason over trajectories, not
only symbolic states.**

The threshold is the key point: the same exploration pattern succeeds at t=8, but becomes impossible once the process reaches health = 0.

Evolution of the model: normal → variant → definitive

This is the central design history: each version solves a limitation of the previous one.



The project moves from symbolic detection, to transport rescue, to autonomous health-based branching. This is principally due to the logical implications of the world “Rescue”.

Validation and demo outputs

The live demo can be replaced by saved planner outputs if the environment fails.

Direct transport

Solved

Plan contains transport actions and does not contain start-stabilize-patient.

Found Plan:
15.0 start-move
21.0 start-inspect
22.0 start-assess
23.0 start-load
33.0 start-unload

Stabilize + transport

Solved

Plan contains start-stabilize-patient before loading and moving the patient.

Found Plan:
10.0 start-assess
11.0 start-stabilize
15.0 start-load
29.0 start-unload

Too-late case

Unsolvable

Forced delay makes the mission fail before rescue can be achieved.

Problem unsolvable
Metric: -1
Dead-Ends detected

Demo path: open final domain + one problem in VS Code, run ENHSP local Docker, then compare with the saved plan file.

Reflection, limitations, and final delivery

The model is deliberately symbolic: it makes planning tractable by making its abstractions explicit.

What works

- Q1 enforces inspect → detect → rescue.
- Q2 shows health/time interaction.
- Definitive validates direct, stabilization, and failure cases.

Known limits

- No real partial observability or belief state.
- No noisy sensing, geometry, motion planning, or execution error.
- Some delayed PDDL+ outputs are documented as planner limitations.

Repository delivery

Code folder:
Codes/...

Report:
Report/report.pdf

Slides:
slide/presentation.pdf

The assignment demonstrates how a planning model can encode information acquisition, time pressure, and health-based branch choice while clearly exposing its abstractions.